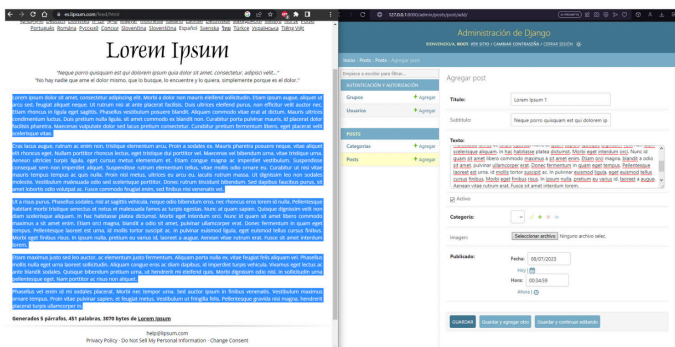


> Pruebas

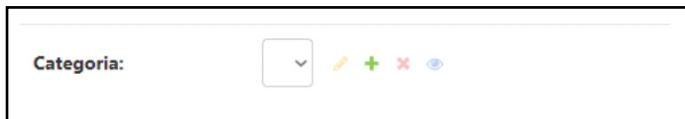
Comenzando a hacer pruebas



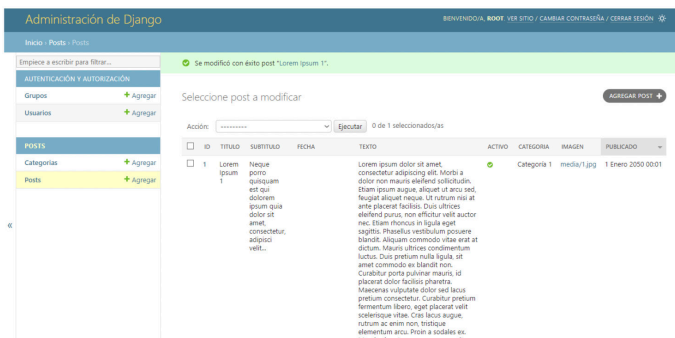
Pasos



Como puedes observar, iremos copiando y pegando para crear contenido. Para la sección de Categorías, haremos click en el símbolo + para añadir una categoría y crearemos una nueva.

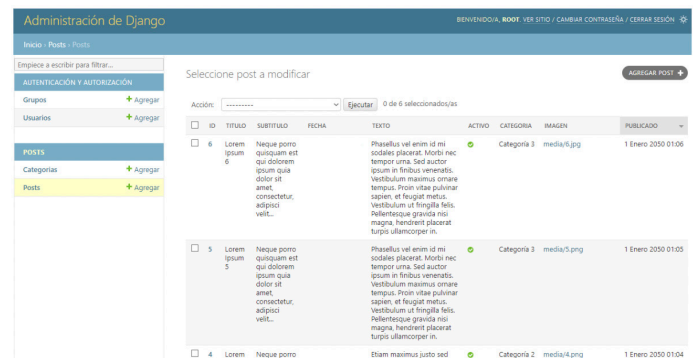


Luego seleccionaremos una imagen de prueba y crearemos nuestro primer artículo con el botón "Guardar".



Verás una pantalla como esta donde saldrá un mensaje en la parte superior, avisándonos que se guardó correctamente.

Vamos a reducir el texto del artículo y crearemos 5 más para las pruebas.

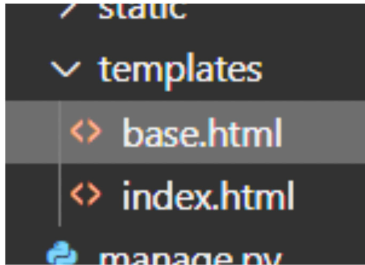


Como puedes observar cada artículo tiene su "id", así como el "título", "subtítulo", "texto", y todas las etiquetas que definimos para que se mostrarán en el "list_display" que definimos previamente en admin.py



Ahora es momento de mostrar estos artículos mediante html, pero primero que nada, vamos a hacer unas modificaciones en el html principal, y para eso vamos a crear un nuevo html que va a contener todas las cosas que el resto de los html también contendrá. Y esto te sonará de algún lado... es porque vamos a usar herencia, pero en este caso se va a heredar de un html a otro html.

Vamos a crear un nuevo archivo html dentro de nuestra carpeta templates con el nombre de base.html.



Dentro de este html usaremos (como lo vimos en páginas anteriores) Bootstrap 5 para generar una estructura html (escribimos bs5, presionamos enter y tendremos nuestra estructura).

> Herencia de html

Modificando htmls



```
base.html
blog > templates > base.html > html > head > title
1 <!doctype html>
2 <html lang="en">
3
4 <head>
5 <title>Title</title>
6 <!-- Required meta tags -->
7 <meta charset="utf-8">
8 <meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no">
9
10 <!-- Bootstrap CSS v5.2.1 -->
11 <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.2.1/dist/css/bootstrap.min.css" rel="stylesheet">
```

Ahora que ya cargamos la estructura base, vamos a comenzar a cargar la etiqueta {% load static %} en la primer línea de nuestro html, que es una etiqueta personalizada de Django que permite cargar los archivos estáticos configurados previamente en settings.py.

Luego vamos a cambiar el lenguaje a "es-ar" para que sea español de Argentina. Más abajo, vamos a cambiar la etiqueta "title" para con la etiqueta de Django {% block title %} Página de inicio {% endblock title %} donde lo que hacemos es que mediante esta etiqueta en cada html que se acceda el título que se ve en la pestaña

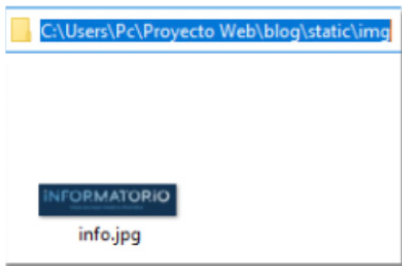
de la página pueda ir cambiando ("Página de inicio" es solo un nombre de referencia, puedes poner el nombre que quieras).

```
base.html
blog > templates > base.html > html
1 {% load static %}
2
3 <!doctype html>
4 <html lang="es-ar">
5
6 <head>
7 <title>{% block title %} Página de inicio {% endblock title %}</title>
8 <!-- Required meta tags -->
9 <meta charset="utf-8">
10 <meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no">
```

Además vamos a crear un navbar genérico con Bootstrap 5 y vamos a comenzar a armar.

Además como para ir organizando y que comience a verse mejor, vamos a agregar una imagen de cabecera. Esto lo vamos a hacer con la etiqueta {% static 'img/info.jpg' %}, donde "img" será la subcarpeta dentro de la carpeta "static", donde vamos a comenzar a poner todos los archivos estáticos, es decir los archivos que no van a cambiar a lo largo del uso de la web. Estos van a

ser la imagen de como para la "marca" de la página, archivos css o js que usaremos, etc. En este caso estamos haciendo referencia a la ruta de la carpeta static --> img ----> y el nombre del archivo (en este caso info) con su extensión (.jpg).



Y si lo vemos en el html (con la estructura para un navbar genérico), quedaría algo así.

```

8  <!-- Required meta tags -->
9  <meta charset="utf-8">
10 <meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no">
11
12 <!-- Bootstrap CSS v5.2.1 -->
13 <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.2.1/dist/css/bootstrap.min.css" rel="stylesheet"
14       integrity="sha384-iYQeCzEYFbKjA/T2uDLTPkwGzCiq6soy8tYaI1GyVh/UjpbCx/TYkiZhlZB6+fzT" crossorigin="anonymous">
15
16 </head>
17
18 <body>
19
20   <header>
21     <a class="flex-sm-fill text-sm-center nav-link active" href="#">
22       
23     </a>
24
25     <nav class="nav nav-pills flex-column flex-sm-row">
26       <a class="flex-sm-fill text-sm-center nav-link active" href="#">Active</a>
27       <a class="flex-sm-fill text-sm-center nav-link" href="#">Longer nav link</a>
28       <a class="flex-sm-fill text-sm-center nav-link" href="#">Link</a>
29       <a class="flex-sm-fill text-sm-center nav-link disabled" href="#" tabindex="-1" aria-disabled="true">Disabled</a>
30     </nav>
31

```

Lo siguiente es configurar el "bloque de contenido", que será la parte del html que irá cambiando según la solicitud http que se pida. Es decir, si vamos a la página de inicio, se mostraría el mismo navbar que en todas las páginas que recorramos, pero cambiaría el contenido principal. Este bloque de contenido lo declararemos en la parte media del html, entre el navbar y el footer (luego haremos el footer), pero recuerda que esto es solo una estructura guía.

Cada uno puede incluir la estructura que desee, pero no te preocupes si aún no entiendes muy bien esta parte, ya se irán aclarando las dudas a medidas que sigamos avanzando. El bloque de contenido lo declararemos usando la etiqueta block:

Si recuerdas en la página 16 y 17 de este apunte, realizamos nuestro primer html llamado "index" en el que mostrábamos el mensaje "Esta es nuestra página de inicio". Además si recuerdas, realizamos una estructura básica para este html, sin embargo, en "base.html" también acabamos de realizar una estructura base con componentes como el Navbar y el Footer que van a ser partes estáticas en todos los html que crearemos. Así que ahora es momento de realizar la herencia de templates o plantillas. En index.html vamos a borrar todo el código que hay dentro y vamos a usar la siguiente etiqueta de Django para hacer la herencia desde base.html a index.html

```
{% block contenido %}
{% endblock %}
```

Crearemos el footer ahora y luego comenzaremos a mostrar lo que estamos haciendo.

```

<> base.html      <> index.html ●
blog > templates > <> index.html
1      {% extends 'base.html' %}
2

```

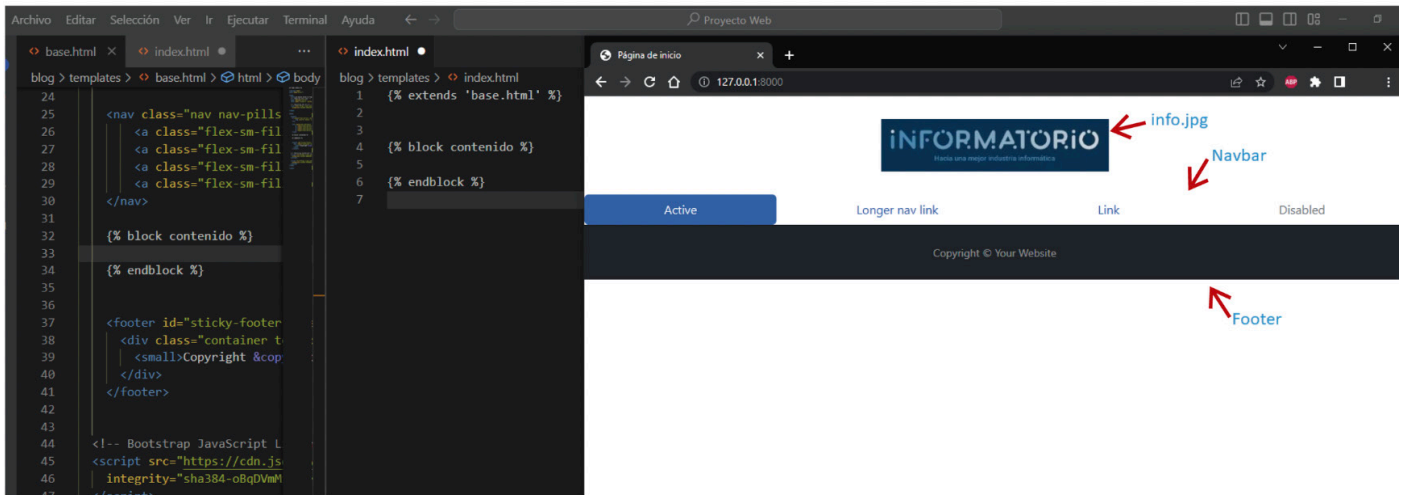
```
{% extends 'base.html' %}
```

Declaramos un Footer genérico.

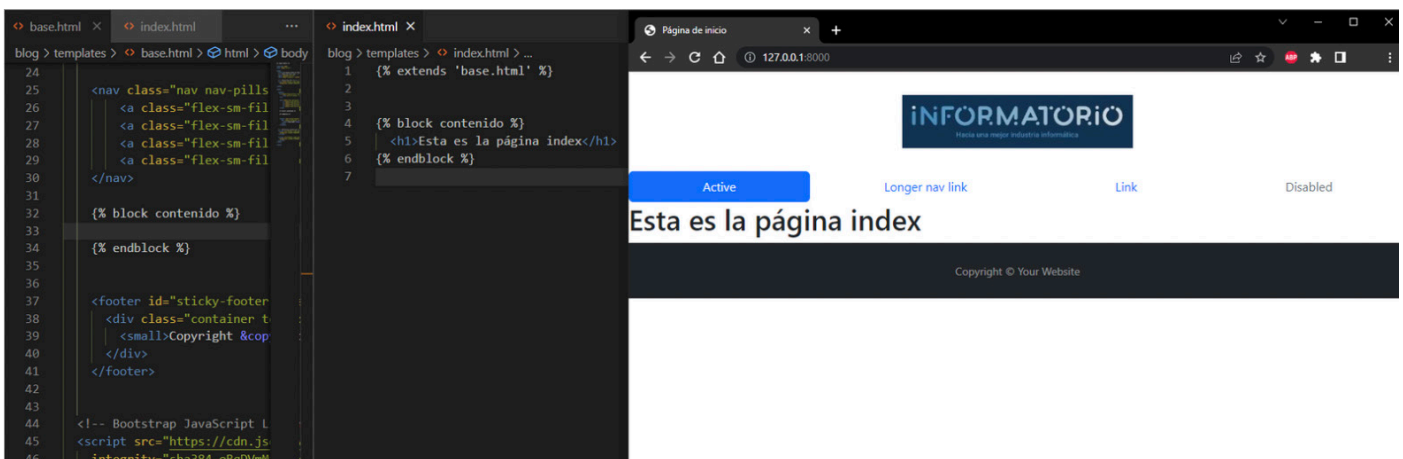
Por último y para finalizar la estructura simple de html que tenemos, van unas líneas de código donde se declaran scripts JS que trae por defecto.

Con esto estamos diciendo a Django que tiene que heredarle a index (o también se podría decir como "extenderle a index") todas las declaraciones en base.html que van a estar fuera de {% block contenido %} y {% endblock %} de esta forma, llamando desde index.html a "block contenido" vamos a poder cargar todo lo que queremos mostrar en el cuerpo de index.html

Te lo mostramos solamente llamando a "block contenido" sin agregar nada y luego agregando algo para que puedas ver la diferencia y lo comprendas mejor:



De lado izquierdo dividimos la consola para que puedas, ver a la vez, el archivo base.html y el archivo index.html, y del lado derecho pusimos la página index para que veás como está armada. Se ve la imagen que pusimos, más abajo el Navbar con los botones por defecto y el Footer. En medio de éstos dos iría la parte del "block contenido", pero como no hay nada en este momento, en esa parte, en index.html, solo se muestra lo que viene por herencia de base.html ¿Pero que pasa si agregamos en index.html?



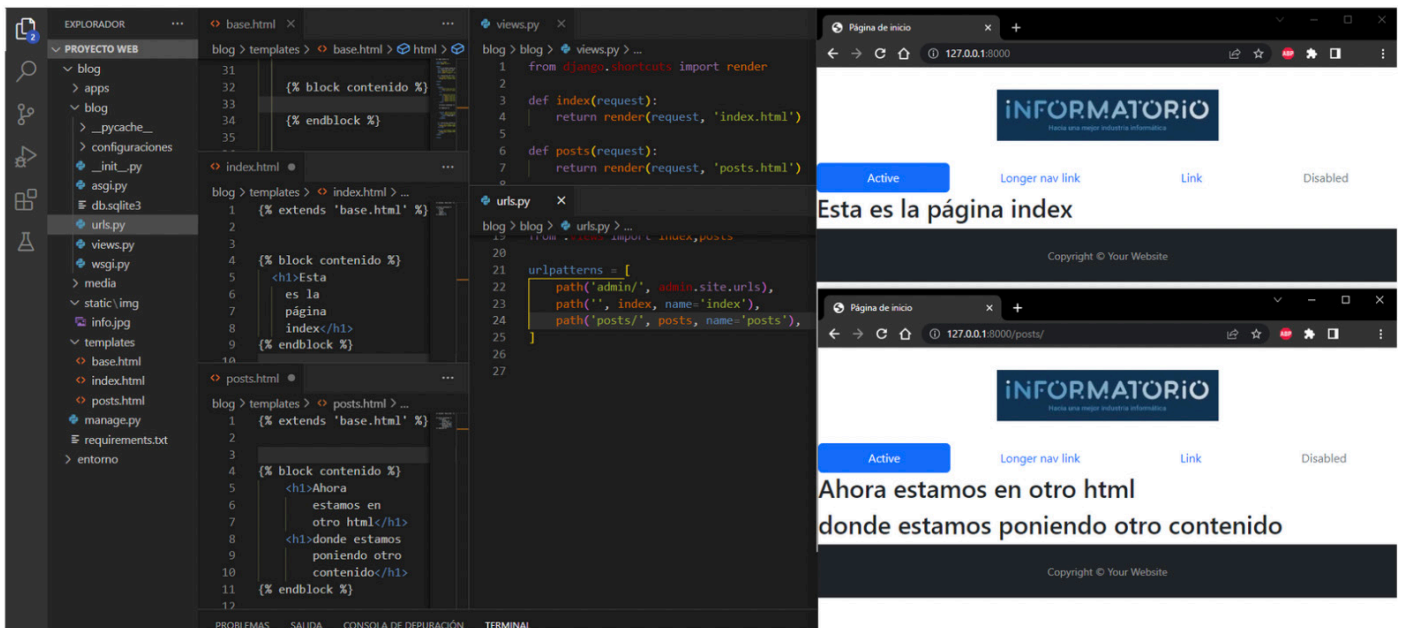
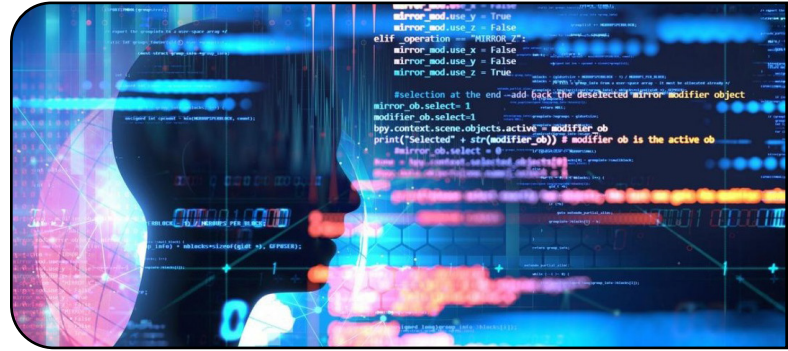
Puedes ver que el contenido que agreguemos en index.html se ve reflejado cuando actualizamos la web. Esto es porque todo lo que se encuentre dentro del "block contenido" que declaramos en base.html pero lo usamos por medio de herencia en index.html, se verá reflejado, mientras que lo que esté fuera del "block contenido" seguirá de la misma forma en todos los html que hereden de base.html.

Para dejarlo más claro, vamos a hacer un nuevo html llamado posts.html donde haremos una herencia, pero pondremos otro mensaje en la parte dinámica (block contenido) para reflejar el cambio de html.

Lo vemos en la siguiente página con el código, además de la creación de la views y url para el otro html que usaremos.

> Views y urls

Declarando views, urls y contenido de html



Parece una pantalla complicada, pero la hicimos así para que sea visualmente lo más completo posible. Te explicamos cada cosa de izquierda a derecha:

- Lo primero que hicimos fue crear otro archivo html: posts.html
- Dentro de posts.html hicimos lo mismo que con index.html, pero cambiamos el contenido dentro del "bloque contenido".
- Después creamos una vista en views.py para poder mostrar el html creado.
- Por último definimos en urls.py la forma de acceso que iba a tener el html --> 127.0.0.1/post

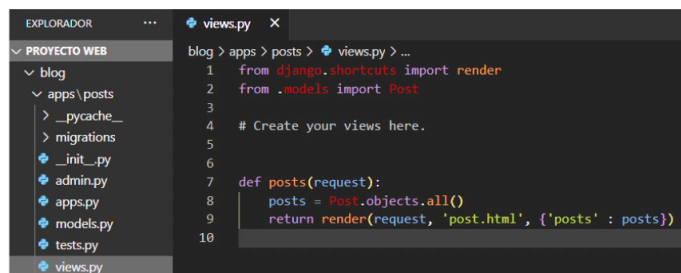
De esta manera podemos hacer toda la herencia que necesitamos, para la cantidad de htmls que necesitamos, sin tener que escribir todo el código inicial de la estructura del html. Esta es una de las características más potentes de Django y la forma que nos deja reutilizar el código.

Ahora podemos comenzar a hacer que se vea un poco mejor visualmente toda la web, pero antes te vamos a mostrar como introducir código Python en Django para poder mostrar los posts como un listado según la cantidad de posts cargados que tengamos en nuestra base de datos.

Puedes observar que la imagen superior, el Navbar y el Footer no cambian, pero si la parte del "block contenido".

Pasos

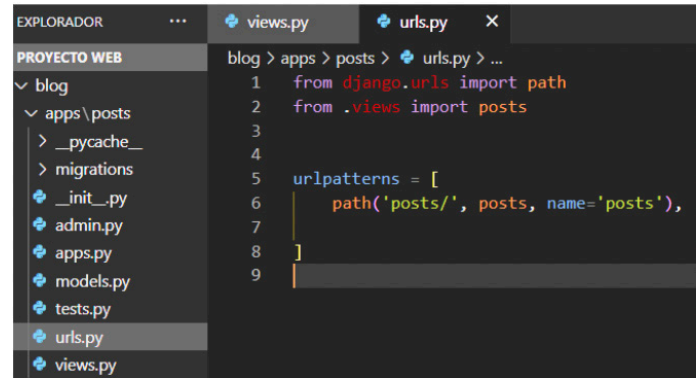
Primero que nada vamos a crear la vista respectiva de "posts" desde la views.py de la aplicación "posts". Como viste antes, la creamos, a modo de ejemplo, desde la views principal, pero ahora vamos a comenzar a ordenar, por lo que crearemos las vistas respectivas desde cada aplicación. Por el momento solo tenemos la aplicación "posts", pero más adelante crearemos otras. Ahora para mostrar los posts cargados en nuestra base de datos vamos a dirigirnos a views.py de "posts" y crearemos una vista basada en funciones en primera instancia, para mostrarte los tipos de vista que se pueden usar (esta vista es la que venimos viendo hasta el momento):



```

EXPLORADOR
PROYECTO WEB
  blog
    apps\posts
      > __pycache__
      > migrations
      > __init__.py
      > admin.py
      > apps.py
      > models.py
      > tests.py
      > views.py
  views.py
1 from django.shortcuts import render
2 from .models import Post
3
4 # Create your views here.
5
6
7 def posts(request):
8     posts = Post.objects.all()
9     return render(request, 'post.html', {'posts': posts})
10
  
```

Lo primero que hicimos fue importar desde "models" el modelo "Post". Luego, definimos la vista basada en función, donde dimos un contexto (artículos), que lo que hace es tomar todos los elementos que se encuentran en "Post" y los trae hacia la vista, que en el "return", aparte de pasarle el html que tiene que mostrar, le pasamos otro parámetro más que es el contexto: {'posts': posts}. Con esto ya estamos extrayendo los registros que tenemos cargados en la base de datos para "Post" en la variable "posts". Lo siguiente es crear la url, pero en nuestra aplicación "posts", sin embargo, este archivo no existe, por lo que vamos a crearlo:



```

EXPLORADOR
PROYECTO WEB
  blog
    apps\posts
      > __pycache__
      > migrations
      > __init__.py
      > admin.py
      > apps.py
      > models.py
      > tests.py
      > urls.py
      > views.py
  urls.py
1 from django.urls import path
2 from .views import posts
3
4
5 urlpatterns = [
6     path('posts/', posts, name='posts'),
7 ]
8
9
  
```

Donde dentro de este archivo urls.py vamos a importar el "path" desde "django.urls" (tal cual en las urls.py principales) y luego traemos la vista "posts" que creamos en views.py

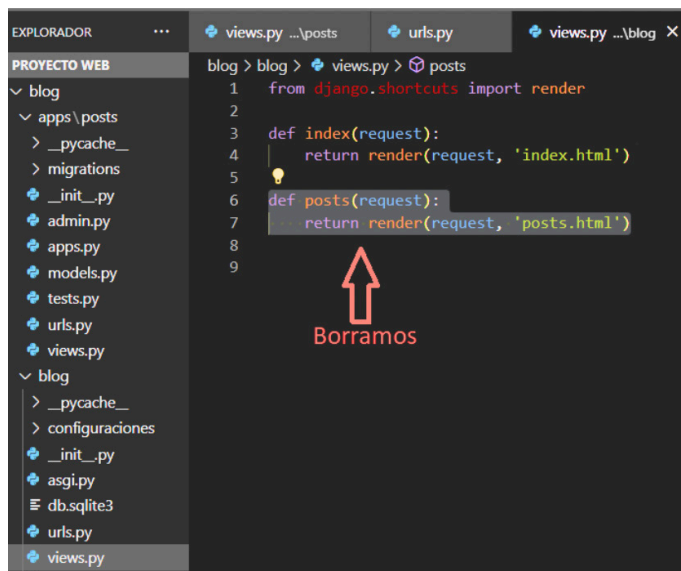
> Views y urls

Declarando views, urls



Pasos

Guardamos los cambios, nos dirigimos a la views.py principal (blog/views.py) y borramos la vista "posts" que tenemos creada ahí ya que la que vamos a utilizar ahora es la de la aplicación.



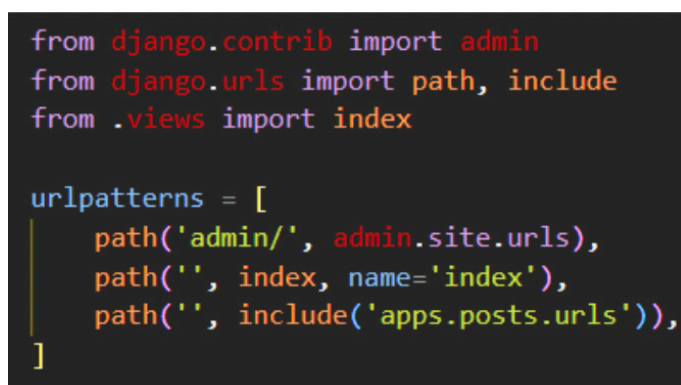
En este urls.py principal, lo que vamos a hacer es importar "include" luego de "path" desde django.urls y vamos a declarar mediante "include" que vamos a utilizar la urls de la aplicación "posts".

Este es el camino que vamos a seguir siempre, ya que las vistas y urls que utilicen nuestras aplicaciones, las crearemos desde las aplicaciones, pero, siempre va a ser necesario, mediante "include", declarar que tenemos que usar la url que se encuentra en nuestra aplicación.

A continuación te mostramos la pantalla dividida de los que son las views.py tanto de la aplicación "posts" como las principales, lo mismo para las urls.

Con esto puedes controlar que todo te haya quedado igual.datos.

Guardamos los cambios y nos dirigimos a el archivo urls.py principal (blog/urls.py) para hacer una modificación ya que desde este archivo, ahora solo tenemos que "avisar" que vamos a utilizar la url de la aplicación "posts".



> Views y urls

Declarando views, urls e introduciendo código python



EXPLORADOR

PROYECTO WEB

blog

apps\posts

__pycache__

migrations

__init__.py

admin.py

apps.py

models.py

tests.py

urls.py

views.py

blog

__pycache__

configuraciones

__init__.py

asgi.py

db.sqlite3

urls.py

views.py

wsgi.py

media

static

templates

views.py

blog > blog > views.py > ...

1

2

3

4

5

6

from django.shortcuts import render

def index(request):

return render(request, 'index.html')

views.py

blog > apps > posts > views.py > ...

1

2

3

4

5

6

7

8

9

10

from django.shortcuts import render

from .models import Post

Create your views here.

def posts(request):

posts = Post.objects.all()

return render(request, 'posts.html', {'posts' : posts})

urls.py

blog > blog > urls.py > ...

16

17

18

19

20

21

22

23

24

25

26

urlpatterns = []

urlpatterns = [

path('admin/', admin.site.urls),

path('', index, name='index'),

path('', include('apps.posts.urls')),

]

urls.py

blog > apps > posts > urls.py > ...

1

2

3

4

5

6

7

8

9

from django.urls import path

from .views import posts

urlpatterns = [

path('posts/', posts, name='posts'),

]

Para no perderte, siempre puedes mirar abajo de la pestaña que está abierta, la ruta de donde se encuentra el archivo abierto. Te lo marcamos en rojo para que lo puedas ver bien.

Una vez hecho todo esto, guardamos todo (nunca debemos olvidarlo, siempre, siempre guardar cada cambio) y ya podemos modificar el html "posts" para que se muestre la lista de registros en nuestra base de datos.

Luego en la imagen más pequeña te dejamos el detalle de la modificación y el uso de Python dentro del código html.

Como puedes ver, utilizamos un "for" tal cual lo usabamos en Python, con una variable que va a hacer la iteración (i) en "posts" que es la variable donde se guardan todos los registros de nuestra base de datos, la cual pasamos como contexto en nuestro html "posts".

La sintaxis es específica de Django con las llaves de apertura y cierre y los signos porcentuales.

posts.html

blog > templates > posts.html > ...

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

{% extends 'base.html' %}

{% block contenido %}

{% for i in posts %}

{{ i.titulo }}

{{ i.subtitulo }}

{{ i.categoria }}

{% empty %}

<h1>No hay registros</h1>

{% endfor %}

<h1>Ahora

estamos en

otro html</h1>

<h1>donde estamos

poniendo otro

contenido</h1>

{% endblock %}

Informatorio Chaco | 2024 | Desarrollo Web

> Código python

Agregando código python en django



Pasos

Luego utilizamos una sentencia más que es "empty" con la misma sintaxis de "for" que lo que hace "empty" es servir de cláusula para que en el caso de que no hayan registros en la base de datos, muestre el texto "No hay posts", ya que si por algún motivo, la base de datos no tuviese ningún registro y no tenemos esta cláusula, la página se mostraría vacía, y eso es un completo error.

Para finalizar usamos "endfor" con la misma sintaxis requerida para Django.

Además has observado que para llamar a los campos de cada registro, utilizamos una sintaxis con doble apertura y cierre de llave, la variable de iteración, luego un punto y seguido, el campo al que estamos llamando.

Podemos llamar a todos los campos que necesitamos, pero a modo de ejemplo, aquí solo llamamos a los campos, "título", "subtitulo" y "categoria".

Las etiquetas
, , son propias de html.

```
{% for i in posts %}

    <br>
    <li>{{ i.titulo }}</li>
    <li>{{ i.subtitulo }}</li>
    <li>{{ i.categoria }}</li>

    {% empty %}
    <h1>No hay registros</h1>

{% endfor %}
```

En este momento, ya podemos hacer algunas modificaciones en nuestros htmls para que se puedan ver mejor, es decir la parte del frontend, ya que básicamente estuvimos trabajando sobre el backend.

Vamos a hacer modificaciones, sin embargo, esto no necesariamente es parte del curso, ya que se está dando en paralelo en las clases de mentoría todo lo que es html, css y js, por lo que solo vamos mostrarte las capturas de pantalla de como quedaría con algunas modificaciones para que se vea más bonito nuestro blog. Usaremos Bootstrap 5 para generar líneas de código html, crearemos un archivo css para estilos y un archivo javascript para dar algunas animaciones.

Esto solo es de ejemplo ya que al finalizar el curso, cada grupo presentará distintas formas de presentación aprendidas en clases, además se podrán descargar plantillas html prediseñadas para dar una mejor visual del trabajo.

A continuación te mostramos nuestras modificaciones de ejemplo para que puedas animarte a hacerlo también.